Retry

Retry · Publish event despite transient broker errors · Build with Waqar

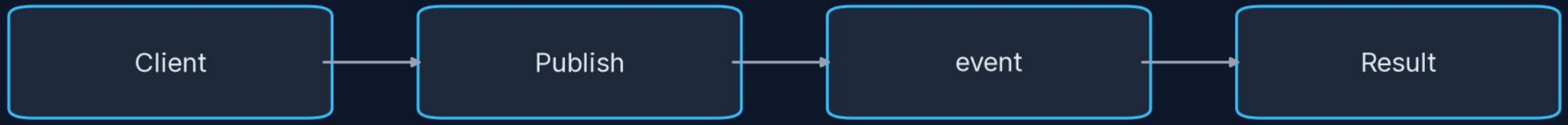
Today's Hook

- Problem: Publish event despite transient broker errors
- Pattern: Retry — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Publish event despite transient broker errors
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Retry: Publish event despite transient broker errors

Real-World Scenario

- Running example: Publish event despite transient broker errors
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.retry.problem;

import java.util.function.Supplier;

public final class NaivePublish {
  public static String once(Supplier<String> s){ return s.get(); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.retry.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class Retry {
  static String call(int maxAttempts, Supplier<String> action){
    RuntimeException last=null;
    for (int i=1;i<=maxAttempts;i++){
      try { return action.get(); } catch (RuntimeException e){ last=e; }
    }
    throw last;
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.retry.problem;

import java.util.concurrent.atomic.AtomicInteger;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Retry: transient broker errors ===");
    System.out.println("--- naive ---");
    try { NaivePublish.once(() -> { throw new RuntimeException("broker"); }); } catch(Exception e){ System.out.println("caught="+e.getMessage()); }
    System.out.println("--- retry ---");
    AtomicInteger n=new AtomicInteger();
    System.out.println(Retry.call(3, () -> { if(n.incrementAndGet()<3) throw new RuntimeException("broker"); return "published"; }));
    try { Retry.call(2, () -> { throw new RuntimeException("still"); }); } catch(RuntimeException e){ System.out.println("caught="+e.getMessage()); }
  }
}
```

Walkthrough Output

```
=== Retry: transient broker errors ===
--- naive ---
caught=broker
--- retry ---
published
caught=still
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Retry.

Related Patterns

- Contrast Retry with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Retry → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.